FINAL DOCUMENTATION

Submitted by:
Elardo, Allaina Marie A.
Matunog, Mateo A.

I. Proposal

Title: SStipend: Budget Tracker

Problem Statement:

Scholars often struggle to manage their limited financial resources due to the lack of efficient tools for tracking expenses and monitoring their Pisay EVC Canteen Card balance. Manual methods are prone to errors and make budgeting difficult. The SStipend: Budget Tracker addresses this problem by automating expense tracking and balance monitoring. Through real-time updates and organized financial summaries, the app simplifies fund management, promotes financial responsibility, and supports informed decision-making among students.

Project Objectives:

1. To design a user-friendly interface that allows scholars to easily record and review their daily expenditures.
2. To automate the monitoring of the Pisay EVC Canteen Card balance and generate real-time updates.
3. To minimize human error in manual expense recording through the use of automation and digital computation.

Features (updated):

1) Automatic File Creation (new)
- Creates input.txt if it does not exist (stores purchase history).
- Creates balance.txt when user sets a balance.

2) Add Expense Entry
- User inputs:
  - Quantity
  - Purchase Unit
  - Description
  - Unit Cost
  - Category

3) Delete Selected Item (new)
- Removes selected row from:
  - Treeview table
  - input.txt file

4) Delete All History (new)
- Confirmation popup appears.
- If confirmed:
    - Clears entire table
    - Clears input.txt

5) Balance Management
- User can:
    - Set initial balance
    - Add additional balance later
- Balance is stored in balance.txt
- Balance updates dynamically on screen

6) Categorizing expenses (new)
- Lets users assign each purchase to a category using a dropdown menu.

7) Expense summary:
1. by category:
- Calculates total spending per category:
    - Food
    - Toiletries
    - Electronics
    - School
    - Miscellaneous
    - Clothing
    - Transportation

Displays results in a new Treeview window.

2. by month:
- Reads stored dates.
- Extracts month from each entry.
- Calculates total expenses per month (January–December).
- Displays results in a Treeview window.

8) Search Feature (new)
- Users can search for specific transactions, and matching results are displayed in a separate table.

9) Edit Purchase Feature
- Users can modify existing entries (quantity, unit, description, price, category).
- Automatically recalculates total cost.
- Updates both the table and input.txt

File/function structure:

This program is a single Python file application.

It interacts with two external text files:

- input.txt: Stores transaction records (one per line, space-separated).
- balance.txt: Stores a single numerical value representing the current user balance.

Functions:

1) *submit():* Validates input, updates the balance file, writes to input.txt, and refreshes the table.
2) *balance_get():* Sets the initial balance for first-time users.
3) *addBalance():* Opens a popup window to input additional funds.
4) *addBalance_Get():* Processes the addition and updates the UI and file.
5) *delete_item():* Removes a specific selected row from the table and the text file.
6) *confirm_delete():* Triggers a confirmation window for clearing all data.
7) *choice_del(opt):* Executes the total wipe of history if "yes" is selected.
8) *summary():* Opens a selection window to choose a summary type.
9) *sum_cat():* Calculates and displays total costs per category in a new window.
10) *sum_month():* Calculates and displays total costs per month in a new window.
11) *info_popup(type):* A dynamic error handler that shows different messages based on the error type.
12) *edit_item():* Opens a window to edit a selected purchase and updates the table.
13) *update_file():* Rewrites input.txt with the updated table data after editing.
14) *search():* Finds and displays records that match a keyword from the search bar.

New Technologies/Tools:

1. *Tkinter (GUI Library):* A built-in Python library for creating graphical user interfaces.
2. *ttk (Themed Tkinter Widgets):* An extension of Tkinter that provides more modern-looking widgets.
3. *File Handling (Text Files as Storage):* Using Python's built-in file system to store and retrieve data.

References:

Python Software Foundation. (2024). Python documentation. https://docs.python.org/3/

Summerfield, M. (2018). Programming in Python 3: A complete introduction to the Python language (2nd ed.). Addison-Wesley Professional.

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

Grayson, J. E. (2000). Python and Tkinter programming. Manning Publications.

Shipman, J. W. (2013). Tkinter 8.5 reference: A GUI for Python. New Mexico Tech. http://www.nmt.edu/tcc/help/pubs/tkinter/

Beazley, D. M. (2009). Python essential reference (4th ed.). Addison-Wesley Professional.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Stallings, W., & Brown, L. (2018). Computer security: Principles and practice (4th ed.). Pearson.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

II. Detailed Methodology

The proposed Budget Tracker system was developed as a standalone desktop application using Python. Both the graphical user interface (GUI) and application logic are integrated within the same environment. Data persistence is handled locally through structured text files, eliminating the need for external servers or cloud-based storage.

A. Technologies Used and Justification:

1) Python: Python was selected due to its readability, extensive standard library, and rapid development capabilities. Its built-in support for file handling, exception management, and GUI integration makes it suitable for small-scale financial management applications.

2) Tkinter & ttk: The graphical interface was implemented using the Tkinter library, with the ttk module for enhanced widget styling. Tkinter was chosen because it is lightweight, cross-platform, and bundled with Python, removing external dependency requirements. The ttk.Treeview widget was used to display structured tabular data, including purchase history and summarized expense reports.

3) File-based storage:
Two text files were utilized:

- input.txt: stores transaction records
- balance.txt: stores the current user balance

The os.path.isfile() method ensures that necessary files are created if absent, preventing runtime errors. File operations were implemented using Python's built-in open() function in read, write, and append modes. This approach was selected for simplicity and ease of deployment, particularly for single-user environments.

B. Implementations of Core functions:

1) *Expense Input and Submission:*

The expense input feature is implemented using Tkinter widgets such as Entry fields and OptionMenu dropdowns, which allow users to enter details like quantity, unit, description, price, and category. These inputs are processed by the submit() function, which retrieves the values and performs validation to ensure they are numeric and logically valid (e.g., not negative or empty). Once validated, the function calculates the total cost and checks it against the available balance to prevent overspending. If all conditions are satisfied, the data is displayed in the Treeview table and simultaneously stored in a text file (input.txt) to ensure persistence.

2) *Data Persistence (File Handling):*

The application uses simple text files (input.txt and balance.txt) to store data permanently. At startup, the program checks whether these files exist using os.path.isfile() and creates them if necessary. Whenever a transaction is added, updated, or deleted, the program reads from or writes to these files to reflect the changes. This approach ensures that all user data, including purchase history and balance, is retained even after the application is closed and reopened.

3) *Table Display (Treeview)*

The recorded expenses are displayed using the ttk.Treeview widget, which provides a structured, table-like interface. Each transaction is shown in columns such as quantity, unit, description, unit price, total cost, category, and date. The table is styled using ttk.Style() to improve readability, including custom header formatting and alternating row colors for better visual distinction. The Treeview serves as the main interface for viewing and interacting with stored data.

4) *Edit Functionality*

The edit feature allows users to modify existing entries in the table. When a row is selected and the edit button is pressed, the edit_item() function opens a new Toplevel window containing input fields pre-filled with the selected data. Users can update the values, and upon saving, the modified data replaces the original row in the Treeview. The update_file() function is then called to rewrite the entire input.txt file, ensuring that the stored data remains consistent with the updated table.

5) *Delete Features*

The application includes both selective and bulk deletion options. The delete_item() function removes a selected row from the Treeview and deletes the corresponding line in the text file by matching the row index. For deleting all records, the confirm_delete() function displays a confirmation dialog, and if the user agrees, the choice_del() function clears both the Treeview and the contents of input.txt. These features ensure that users have full control over managing their data.

6) *Balance Management*

The balance management system tracks the user's available funds and ensures that spending does not exceed this amount. If no balance file exists, the user is prompted to input an initial balance, which is then stored in balance.txt. The balance is updated whenever a purchase is made or when additional funds are added through the "Add Balance" feature. The program continuously reads from and writes to the balance file to maintain an accurate and up-to-date value displayed in the interface.

7) *Search Feature*

The search functionality enables users to locate specific transactions quickly. The search() function reads all entries from input.txt and checks each line for the presence of a

user-provided keyword. Matching results are stored in a list and displayed in a new Toplevel window using another Treeview table. This implementation uses simple substring matching, making it flexible for searching across different fields such as description or category.

8) *Summary of Expenses*

The summary feature provides insights into spending patterns by grouping expenses either by category or by month. When activated, the program reads all entries from input.txt and uses loops and conditional statements to accumulate totals based on the selected grouping. The results are then displayed in a new Treeview table. This feature allows users to analyze their spending habits and identify trends over time.

9) *Input Validation and Error Handling*

To ensure data integrity, the application incorporates input validation using try/except blocks and conditional checks. Invalid inputs such as non-numeric values, negative numbers, or missing selections trigger error messages displayed through the info_popup() function. These popups provide clear feedback to the user, guiding them to correct their input before proceeding.

10) *UI Design and Layout*

The overall interface design uses a consistent color scheme (green and yellow) and organized layout to enhance readability and usability. Widgets are positioned using the .place() method for precise control over their location. The interface is divided into logical sections, including input fields, action buttons, and the data table. Styling choices such as color themes and font settings contribute to a clean and visually appealing user experience.

C. Backend–Frontend Interaction:

The application operates without network communication. Backend logic and frontend components interact through:

- Widget value retrieval (Entry.get(), StringVar.get())
- Event-driven callbacks attached to buttons
- Dynamic updates to Treeview widgets

All operations are executed locally and synchronously.

D. Design Decisions and Trade-Offs:

1) Predefined expense categories were used to standardize classification and simplify aggregation logic.
Trade-off: Users cannot dynamically define new categories, limiting customization.

2) Global variables were utilized for summary computations and Toplevel window references to simplify cross-functional data sharing.
Trade-off: While this reduces parameter passing complexity, it may reduce modularity and scalability in larger systems.

E. Ethical and Responsible Programming Considerations:

1) User Privacy: The system operates entirely offline, ensuring that financial data is not transmitted externally. No third-party services are integrated, protecting user spending information from external exposure.

2) Data Integrity and Error Prevention: Input validation mechanisms prevent invalid financial entries. Exception handling prevents system crashes due to incorrect user input. Confirmation dialogs reduce accidental deletion of financial records. These design choices prioritize user safety and reliability.